

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: MLA03
Environment and Agent

COURSE NAME: Reinforcement learning for

COURSE OUTCOME COVERED

***CO3:** Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments.(BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Experiments

Duration: 3hrs

-
1. **Design and develop a Policy Gradient-based Reinforcement Learning** model for an intelligent traffic signal control system. Implement the solution using Python and TensorFlow/PyTorch, compare **REINFORCE** and **A2C**, and evaluate average vehicle waiting time, throughput, and convergence rate.

```
A.import numpy as np
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
import matplotlib.pyplot as plt
```

```
class TrafficEnvironment:
```

```
    def __init__(self):
```

```

self.max_steps = 50

self.reset()

def reset(self):

    # Vehicles waiting in two directions

    self.north_south = np.random.randint(5, 15)

    self.east_west = np.random.randint(5, 15)

    self.step_count = 0

    self.total_waiting = 0

    self.total_throughput = 0

    return self.get_state()

def get_state(self):

    # Normalize vehicle counts

    return np.array([

        self.north_south / 20.0,

        self.east_west / 20.0

    ], dtype=np.float32)

def step(self, action):

    # Action:

    # 0 = Green for North-South

    # 1 = Green for East-West

    if action == 0:

        passed = min(self.north_south, 5)

        self.north_south -= passed

        # Vehicles continue arriving

        self.north_south += np.random.randint(0, 4)

```

```

        self.east_west += np.random.randint(0, 4)
    else:
        passed = min(self.east_west, 5)
        self.east_west -= passed
        self.north_south += np.random.randint(0, 4)
        self.east_west += np.random.randint(0, 4)

    # Waiting vehicles
    waiting = self.north_south + self.east_west
    self.total_waiting += waiting
    self.total_throughput += passed

    # Reward:

    # Less waiting = better reward
    # More vehicles passed = better reward
    reward = passed - 0.2 * waiting
    self.step_count += 1

    done = self.step_count >= self.max_steps
    return self.get_state(), reward, done

```

2. POLICY NETWORK

```

class PolicyNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(2, 32),
            nn.ReLU(),
            nn.Linear(32, 2),

```

```

        nn.Softmax(dim=-1)

    )

    def forward(self, state):

        return self.network(state)

# 3. VALUE NETWORK FOR A2C

class ValueNetwork(nn.Module):

    def __init__(self):

        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(2, 32),

            nn.ReLU(),

            nn.Linear(32, 1)

        )

    def forward(self, state):

        return self.network(state)

# 4. REINFORCE ALGORITHM

def train_reinforce(episodes=300, gamma=0.99):

    env = TrafficEnvironment()

    policy = PolicyNetwork()

    optimizer = optim.Adam(

        policy.parameters(),

        lr=0.001

    )

    episode_rewards = []

    waiting_times = []

```

```

throughputs = []

for episode in range(epochs):

    state = env.reset()

    log_probs = []

    rewards = []

    done = False

    while not done:

        state_tensor = torch.tensor(

            state,

            dtype=torch.float32

        )

        probabilities = policy(state_tensor)

        distribution = torch.distributions.Categorical(

            probabilities

        )

        action = distribution.sample()

        log_prob = distribution.log_prob(action)

        next_state, reward, done = env.step(

            action.item()

        )

        log_probs.append(log_prob)

        rewards.append(reward)

        state = next_state

    # Calculate discounted returns

```

```

returns = []

G = 0

for reward in reversed(rewards):

    G = reward + gamma * G

    returns.insert(0, G)

returns = torch.tensor(

    returns,

    dtype=torch.float32

)

# Normalize returns

returns = (

    returns - returns.mean()

) / (returns.std() + 1e-8)

loss = 0

for log_prob, G in zip(

    log_probs,

    returns

):

    loss += -log_prob * G

optimizer.zero_grad()

loss.backward()

optimizer.step()

episode_rewards.append(sum(rewards))

waiting_times.append(

    env.total_waiting / env.max_steps

```

```

    )

    throughputs.append(
        env.total_throughput
    )

    return (
        episode_rewards,
        waiting_times,
        throughputs
    )

```

5. A2C ALGORITHM

```

def train_a2c(episodes=300, gamma=0.99):

    env = TrafficEnvironment()

    policy = PolicyNetwork()

    value = ValueNetwork()

    policy_optimizer = optim.Adam(
        policy.parameters(),
        lr=0.001
    )

    value_optimizer = optim.Adam(
        value.parameters(),
        lr=0.001
    )

    episode_rewards = []

    waiting_times = []

    throughputs = []

```

```
for episode in range(epochs):

    state = env.reset()

    log_probs = []

    values = []

    rewards = []

    done = False

    while not done:

        state_tensor = torch.tensor(

            state,

            dtype=torch.float32

        )

        probabilities = policy(

            state_tensor

        )

        distribution = torch.distributions.Categorical(

            probabilities

        )

        action = distribution.sample()

        log_prob = distribution.log_prob(

            action

        )

        state_value = value(

            state_tensor

        ).squeeze()
```



```

        next_state, reward, done = env.step(
            action.item()
        )

        log_probs.append(log_prob)

        values.append(state_value)

        rewards.append(reward)

        state = next_state

# Calculate returns
returns = []

G = 0

for reward in reversed(rewards):

    G = reward + gamma * G

    returns.insert(0, G)

returns = torch.tensor(
    returns,
    dtype=torch.float32
)

values_tensor = torch.stack(values)

log_probs_tensor = torch.stack(
    log_probs
)

# Advantage
advantage = (
    returns - values_tensor.detach()
)

```

```

# Actor loss

actor_loss = -(
    log_probs_tensor * advantage
).mean()

# Critic loss

critic_loss = (
    returns - values_tensor
).pow(2).mean()

# Update policy

policy_optimizer.zero_grad()

actor_loss.backward()

policy_optimizer.step()

# Update value network

value_optimizer.zero_grad()

critic_loss.backward()

value_optimizer.step()

episode_rewards.append(
    sum(rewards)
)

waiting_times.append(
    env.total_waiting / env.max_steps
)

throughputs.append(
    env.total_throughput
)

```

```

return (
    episode_rewards,
    waiting_times,
    throughputs
)

# 6. TRAIN BOTH MODELS

print("Training REINFORCE...")

reinforce_rewards, reinforce_waiting, reinforce_throughput = \
    train_reinforce()

print("REINFORCE Training Completed.")

print("\nTraining A2C...")

a2c_rewards, a2c_waiting, a2c_throughput = \
    train_a2c()

print("A2C Training Completed.")

# 7. CALCULATE PERFORMANCE

def average_last_episodes(data, n=50):
    return np.mean(data[-n:])

reinforce_avg_reward = average_last_episodes(
    reinforce_rewards
)

a2c_avg_reward = average_last_episodes(
    a2c_rewards
)

reinforce_avg_waiting = average_last_episodes(
    reinforce_waiting

```

```

)

a2c_avg_waiting = average_last_episodes(

    a2c_waiting

)

reinforce_avg_throughput = average_last_episodes(

    reinforce_throughput

)

a2c_avg_throughput = average_last_episodes(

    a2c_throughput

)

# 8. DISPLAY COMPARISON TABLE

print("\n=====")

print("    PERFORMANCE COMPARISON")

print("=====")

print(

    f'{'Metric':<25}'

    f'{'REINFORCE':<15}'

    f'{'A2C':<15}'

)

print("-" * 55)

print(

    f'{'Average Reward':<25}'

    f'{'reinforce_avg_reward':<15.2f}'

    f'{'a2c_avg_reward':<15.2f}'

)

```

```
print(
    f'{"Average Waiting Time":<25}'
    f'{"reinforce_avg_waiting":<15.2f}'
    f'{"a2c_avg_waiting":<15.2f}'
)
```

```
print(
    f'{"Average Throughput":<25}'
    f'{"reinforce_avg_throughput":<15.2f}'
    f'{"a2c_avg_throughput":<15.2f}'
)
```

9. TRAINING REWARD GRAPH

```
plt.figure(figsize=(10, 5))

plt.plot(
    reinforce_rewards,
    label="REINFORCE"
)

plt.plot(
    a2c_rewards,
    label="A2C"
)

plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.title("REINFORCE vs A2C - Training Reward")
plt.legend()
plt.grid()
```

```
plt.show()

# 10. WAITING TIME GRAPH

plt.figure(figsize=(10, 5))

plt.plot(

    reinforce_waiting,

    label="REINFORCE"

)

plt.plot(

    a2c_waiting,

    label="A2C"

)

plt.xlabel("Episode")

plt.ylabel("Average Waiting Time")

plt.title(

    "REINFORCE vs A2C - Vehicle Waiting Time"

)

plt.legend()

plt.grid()

plt.show()

# 11. THROUGHPUT GRAPH

plt.figure(figsize=(10, 5))

plt.plot(

    reinforce_throughput,

    label="REINFORCE"

)
```

```
plt.plot(
    a2c_throughput,
    label="A2C"
)

plt.xlabel("Episode")

plt.ylabel("Vehicles Passed")

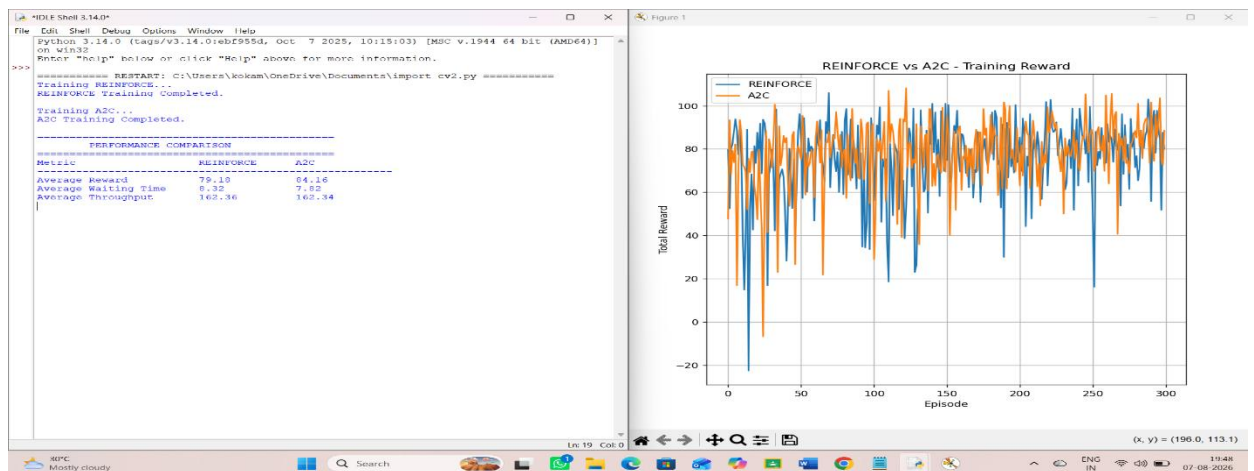
plt.title(
    "REINFORCE vs A2C - Traffic Throughput"
)

plt.legend()

plt.grid()

plt.show()

OUTPUT;
```



(10 Marks)

2. Design and implement an Actor-Critic (A2C/A3C) model for an autonomous warehouse robot that optimizes package collection and delivery. Compare the performance of **Vanilla Policy Gradient** and **Actor-Critic** algorithms based on reward, training stability, and task completion time.

A.import numpy as np

import torch

```
import torch.nn as nn

import torch.optim as optim

import matplotlib.pyplot as plt

# 1. WAREHOUSE ENVIRONMENT

class WarehouseEnv:

    def __init__(self, size=5):

        self.size = size

        self.max_steps = 50

    def reset(self):

        self.robot = [0, 0]

        self.package = [2, 2]

        self.goal = [4, 4]

        self.package_collected = False

        self.steps = 0

        return self.get_state()

    def get_state(self):

        return np.array([

            self.robot[0] / (self.size - 1),

            self.robot[1] / (self.size - 1),

            self.package[0] / (self.size - 1),

            self.package[1] / (self.size - 1),

            self.goal[0] / (self.size - 1),

            self.goal[1] / (self.size - 1),

            float(self.package_collected)

        ], dtype=np.float32)
```



```

def step(self, action):

    # 0 = Up

    # 1 = Down

    # 2 = Left

    # 3 = Right

    old_position = self.robot.copy()

    if action == 0:

        self.robot[0] = max(0, self.robot[0] - 1)

    elif action == 1:

        self.robot[0] = min(

            self.size - 1,

            self.robot[0] + 1

        )

    elif action == 2:

        self.robot[1] = max(0, self.robot[1] - 1)

    elif action == 3:

        self.robot[1] = min(

            self.size - 1,

            self.robot[1] + 1

        )

    self.steps += 1

    # Basic movement penalty

    reward = -0.1

    # Extra penalty if robot hits boundary

    if self.robot == old_position:

```

```

        reward -= 0.5

# Collect package
if (
    not self.package_collected
    and self.robot == self.package
):
    self.package_collected = True
    reward += 10

# Deliver package
if (
    self.package_collected
    and self.robot == self.goal
):
    reward += 20
    done = True

    return (
        self.get_state(),
        reward,
        done
    )

done = self.steps >= self.max_steps

return (
    self.get_state(),
    reward,
    done

```

)

2. POLICY NETWORK

```
class PolicyNetwork(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.network = nn.Sequential(
```

```
            nn.Linear(7, 64),
```

```
            nn.ReLU(),
```

```
            nn.Linear(64, 4),
```

```
            nn.Softmax(dim=-1)
```

```
        )
```

```
    def forward(self, state):
```

```
        return self.network(state)
```

3. VALUE NETWORK

```
class ValueNetwork(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.network = nn.Sequential(
```

```
            nn.Linear(7, 64),
```

```
            nn.ReLU(),
```

```
            nn.Linear(64, 1)
```

```
        )
```

```
    def forward(self, state):
```

```
        return self.network(state)
```

4. VANILLA POLICY GRADIENT / REINFORCE

```

def train_reinforce(
    episodes=400,
    gamma=0.99):
    env = WarehouseEnv()
    policy = PolicyNetwork()
    optimizer = optim.Adam(
        policy.parameters(),
        lr=0.001
    )
    rewards_history = []
    completion_times = []
    for episode in range(episodes):
        state = env.reset()
        log_probs = []
        rewards = []
        done = False
        while not done:
            state_tensor = torch.tensor(
                state,
                dtype=torch.float32
            )
            probabilities = policy(
                state_tensor
            )

```

```

distribution = torch.distributions.Categorical(
    probabilities
)
action = distribution.sample()
log_prob = distribution.log_prob(
    action
)
next_state, reward, done = env.step(
    action.item()
)
log_probs.append(log_prob)
rewards.append(reward)
state = next_state
# Calculate discounted returns
returns = []
G = 0
for reward in reversed(rewards):
    G = reward + gamma * G
    returns.insert(0, G)
returns = torch.tensor(
    returns,
    dtype=torch.float32
)
# Normalize returns
if len(returns) > 1:

```

```

        returns = (
            returns - returns.mean()
        ) / (returns.std() + 1e-8)

# Policy loss

loss = 0

for log_prob, G in zip(
    log_probs,
    returns):

    loss += -log_prob * G

optimizer.zero_grad()

loss.backward()

optimizer.step()

rewards_history.append(
    sum(rewards)
)

completion_times.append(
    env.steps
)

return rewards_history, completion_times

# 5. ACTOR-CRITIC (A2C)

def train_a2c(
    episodes=400,
    gamma=0.99):

    env = WarehouseEnv()

    actor = PolicyNetwork()

```

```

critic = ValueNetwork()

actor_optimizer = optim.Adam(
    actor.parameters(),
    lr=0.001
)

critic_optimizer = optim.Adam(
    critic.parameters(),
    lr=0.001
)

rewards_history = []
completion_times = []

for episode in range(episodes):
    state = env.reset()

    log_probs = []
    values = []
    rewards = []

    done = False

    while not done:
        state_tensor = torch.tensor(
            state,
            dtype=torch.float32
        )

        probabilities = actor(
            state_tensor
        )

```

```

distribution = torch.distributions.Categorical(
    probabilities
)
action = distribution.sample()
log_prob = distribution.log_prob(
    action
)
state_value = critic(
    state_tensor
).squeeze()
next_state, reward, done = env.step(
    action.item()
)
log_probs.append(log_prob)
values.append(state_value)
rewards.append(reward)
state = next_state
# Calculate discounted returns
returns = []
G = 0
for reward in reversed(rewards):
    G = reward + gamma * G
    returns.insert(0, G)
returns = torch.tensor(
    returns,

```



```

        dtype=torch.float32
    )

    values_tensor = torch.stack(values)

    log_probs_tensor = torch.stack(log_probs)

    # Advantage

    advantage = (

        returns - values_tensor.detach()

    )

    # Actor loss

    actor_loss = -(

        log_probs_tensor * advantage

    ).mean()

    # Critic loss

    critic_loss = (

        returns - values_tensor

    ).pow(2).mean()

    # Update Actor

    actor_optimizer.zero_grad()

    actor_loss.backward()

    actor_optimizer.step()

    # Update Critic

    critic_optimizer.zero_grad()

    critic_loss.backward()

    critic_optimizer.step()

    rewards_history.append(

```

```

        sum(rewards)
    )

    completion_times.append(
        env.steps
    )

    return rewards_history, completion_times

# 6. TRAIN BOTH MODELS

print("Training Vanilla Policy Gradient...")

reinforce_rewards, reinforce_times = \
    train_reinforce()

print("REINFORCE training completed.")

print("\nTraining Actor-Critic A2C...")

a2c_rewards, a2c_times = \
    train_a2c()

print("A2C training completed.")

# 7. PERFORMANCE CALCULATION

last_n = 50

reinforce_avg_reward = np.mean(
    reinforce_rewards[-last_n:]
)

a2c_avg_reward = np.mean(
    a2c_rewards[-last_n:]
)

reinforce_avg_time = np.mean(
    reinforce_times[-last_n:]
)

```

```

)

a2c_avg_time = np.mean(

    a2c_times[-last_n:]

)

# Standard deviation = training stability measure

reinforce_stability = np.std(

    reinforce_rewards[-last_n:]

)

a2c_stability = np.std(

    a2c_rewards[-last_n:]

)

# 8. COMPARISON TABLE

print("\n=====")

print("    REINFORCE vs A2C COMPARISON")

print("=====")

print(

    f'{'Metric':<30}'

    f'{'REINFORCE':<15}'

    f'{'A2C':<15}'

)

print("-" * 60)

print(

    f'{'Average Reward':<30}'

    f'{'reinforce_avg_reward:<15.2f}'

    f'{'a2c_avg_reward:<15.2f}'

```

```

)

print(
    f'{'Reward Std. Deviation':<30}'
    f'{'reinforce_stability':<15.2f}'
    f'{'a2c_stability':<15.2f}'
)

print(
    f'{'Average Task Time':<30}'
    f'{'reinforce_avg_time':<15.2f}'
    f'{'a2c_avg_time':<15.2f}'
)

)

# 9. REWARD GRAPH

plt.figure(figsize=(10, 5))

plt.plot(
    reinforce_rewards,
    label="REINFORCE"
)

plt.plot(
    a2c_rewards,
    label="A2C"
)

plt.xlabel("Episode")

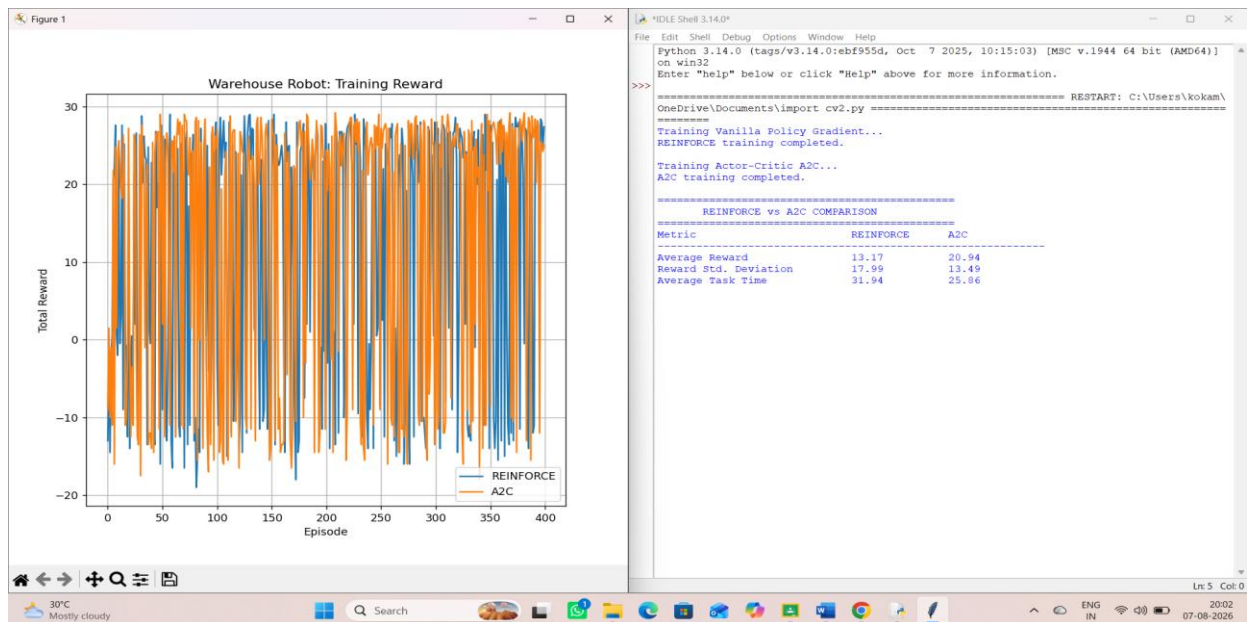
plt.ylabel("Total Reward")

plt.title(
    "Warehouse Robot: Training Reward"

```

```
)  
plt.legend()  
plt.grid()  
plt.show()  
  
# 10. TASK COMPLETION TIME GRAPH  
  
plt.figure(figsize=(10, 5))  
  
plt.plot(  
    reinforce_times,  
    label="REINFORCE"  
)  
  
plt.plot(  
    a2c_times,  
    label="A2C"  
)  
  
plt.xlabel("Episode")  
plt.ylabel("Steps to Completion")  
plt.title(  
    "Warehouse Robot: Task Completion Time"  
)  
  
plt.legend()  
plt.grid()  
plt.show()
```

OUTPUT;



(10 Marks)

3. Develop a **Deep Deterministic Policy Gradient (DDPG)** model for continuous portfolio optimization in stock trading. Design the state space, action space, reward function, and evaluate cumulative return, risk, and convergence performance.
- (10 Marks)

```
A.import numpy as np

import torch

import torch.nn as nn

import torch.optim as optim

import matplotlib.pyplot as plt

class PortfolioEnv:

    def __init__(self):

        self.max_steps = 50

        self.reset()

    def reset(self):

        self.current_step = 0

        self.portfolio_value = 1000.0
```

```

# Initial state: returns of 3 stocks

self.state = np.array(

    [0.01, 0.02, 0.015],

    dtype=np.float32

)

return self.state

def step(self, action):

    # Make action positive

    action = np.abs(action)

    # Convert action into portfolio weights

    total = np.sum(action)

    if total == 0:

        action = np.ones(3) / 3

    else:

        action = action / total

    # Simulated returns for 3 stocks

    returns = np.random.normal(

        [0.01, 0.015, 0.008],

        [0.02, 0.03, 0.015]

    )

    # Calculate portfolio return

    portfolio_return = np.dot(

        action,

        returns

```

```

)

# Calculate risk

risk = np.std(returns)

# Reward

reward = portfolio_return - 0.1 * risk

# Update portfolio value

self.portfolio_value *= (

    1 + portfolio_return

)

self.current_step += 1

# Check whether episode is finished

done = self.current_step >= self.max_steps

# New state

next_state = returns.astype(

    np.float32

)

self.state = next_state

return (

    next_state,

    float(reward),

    done,

    float(risk)

)

```



```

class Actor(nn.Module):

    def __init__(self):
        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(3, 32),

            nn.ReLU(),

            nn.Linear(32, 32),

            nn.ReLU(),

            nn.Linear(32, 3),

            nn.Softmax(dim=1)

        )

    def forward(self, state):

        return self.network(state)

class Critic(nn.Module):

    def __init__(self):
        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(6, 32),

            nn.ReLU(),

            nn.Linear(32, 32),

            nn.ReLU(),

            nn.Linear(32, 1)

        )

    def forward(self, state, action):

        x = torch.cat(

```

```

        [state, action],
        dim=1
    )

    return self.network(x)

env = PortfolioEnv()

actor = Actor()

critic = Critic()

actor_optimizer = optim.Adam(
    actor.parameters(),
    lr=0.001
)

critic_optimizer = optim.Adam(
    critic.parameters(),
    lr=0.001
)

episodes = 100

reward_history = []

portfolio_history = []

risk_history = []

print("DDPG Portfolio Optimization")

print("=====")

print("Training started...\n")


for episode in range(episodes):

    state = env.reset()

```

```
total_reward = 0

total_risk = 0

done = False

while not done:

    # Convert state to tensor

    state_tensor = torch.tensor(

        state,

        dtype=torch.float32

    ).unsqueeze(0)

    # Actor selects portfolio weights

    action = actor(

        state_tensor

    )

    # Add exploration noise

    noise = torch.randn_like(action) * 0.05

    noisy_action = torch.clamp(

        action + noise,

        0,

        1

    )

    # Environment step

    (

        next_state,

        reward,
```

```
        done,

        risk

    ) = env.step(

        noisy_action.detach().numpy()[0]

    )

    # Convert reward to tensor

    reward_tensor = torch.tensor(

        [[reward]],

        dtype=torch.float32

    )

    predicted_value = critic(

        state_tensor,

        noisy_action

    )

    critic_loss = (

        predicted_value - reward_tensor

    ).pow(2).mean()

    critic_optimizer.zero_grad()

    critic_loss.backward()

    critic_optimizer.step()

    new_action = actor(

        state_tensor
```

```

    )

    actor_loss = -critic(

        state_tensor,

        new_action

    ).mean()

    actor_optimizer.zero_grad()

    actor_loss.backward()

    actor_optimizer.step()

    # Update values

    state = next_state

    total_reward += reward

    total_risk += risk


average_risk = (

    total_risk / env.max_steps

)

reward_history.append(

    total_reward

)

portfolio_history.append(

    env.portfolio_value

)

risk_history.append(

    average_risk

)

```

```

# Display progress

if (episode + 1) % 10 == 0:

    print(

        "Episode:",

        episode + 1,

        "| Reward:",

        round(total_reward, 4),

        "| Portfolio Value:",

        round(env.portfolio_value, 2),

        "| Risk:",

        round(average_risk, 4)

    )

print("\n=====")

print("Training Completed")

print("=====")

print(

    "Final Portfolio Value:",

    round(portfolio_history[-1], 2)

)

print(

    "Average Reward:",

    round(

        np.mean(reward_history[-10:]),

        4

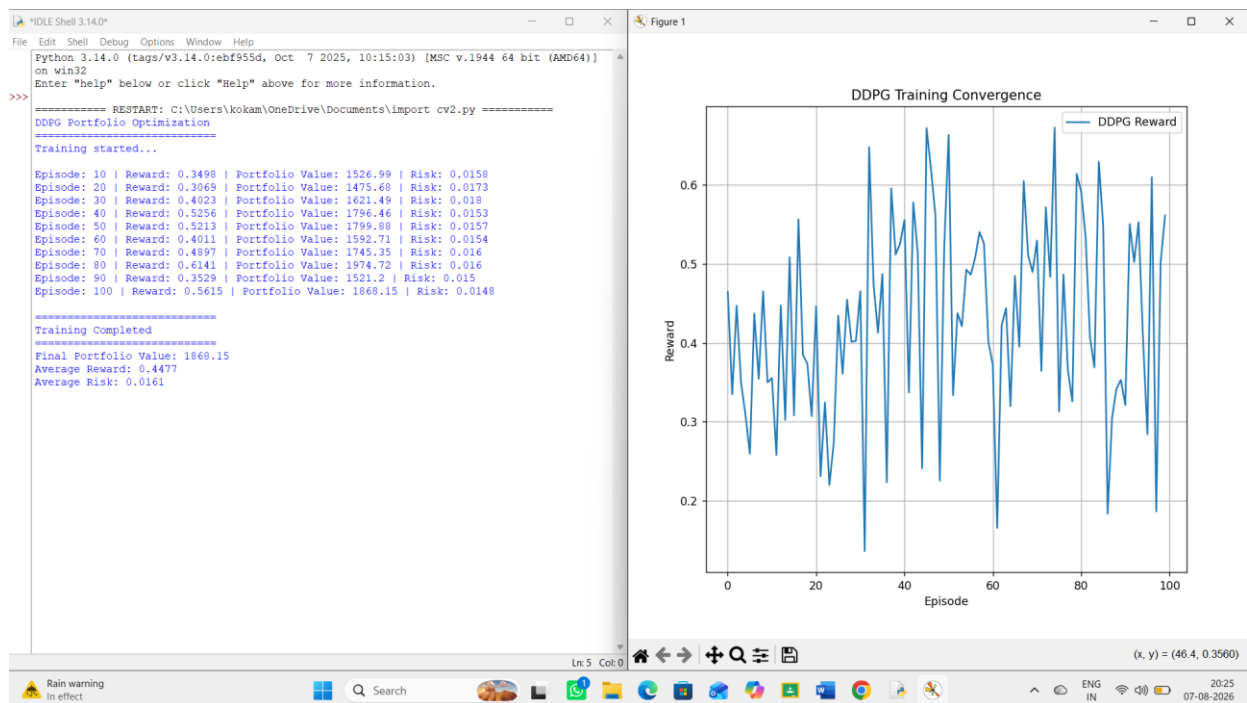
    )

```

```
)  
print(  
    "Average Risk:",  
    round(  
        np.mean(risk_history[-10:]),  
        4  
    )  
)  
  
plt.figure(figsize=(8, 5))  
plt.plot(  
    reward_history,  
    label="DDPG Reward"  
)  
  
plt.xlabel("Episode")  
plt.ylabel("Reward")  
plt.title(  
    "DDPG Training Convergence"  
)  
  
plt.legend()  
plt.grid()  
plt.show()  
  
plt.figure(figsize=(8, 5))  
plt.plot(  
    portfolio_history,  
    label="Portfolio Value"
```

```
)  
  
plt.xlabel("Episode")  
  
plt.ylabel("Portfolio Value")  
  
plt.title(  
    "Cumulative Portfolio Performance"  
)  
  
plt.legend()  
  
plt.grid()  
  
plt.show()  
  
plt.figure(figsize=(8, 5))  
  
plt.plot(  
    risk_history,  
    label="Portfolio Risk"  
)  
  
plt.xlabel("Episode")  
  
plt.ylabel("Risk")  
  
plt.title(  
    "Portfolio Risk"  
)  
  
plt.legend()  
  
plt.grid()  
  
plt.show()
```


OUTPUT;



4. **Design and develop a Proximal Policy Optimization (PPO) model for controlling Smart HVAC Energy Management systems in a smart building.** Implement the model to minimize energy consumption while maintaining occupant comfort, and compare PPO with REINFORCE.

```
A.import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
```

```
class HVACEnv:
```

```
    def __init__(self):
        self.max_steps = 50
        self.target_temperature = 24.0

    def reset(self):
        self.temperature = 30.0
        self.outdoor_temperature = 32.0
        self.occupancy = 5
        self.step_count = 0
        self.energy = 0
        return self.get_state()
    def get_state(self):

        return np.array([
```

```

        self.temperature / 40,
        self.outdoor_temperature / 40,
        self.occupancy / 10
    ], dtype=np.float32)

```

```

def step(self, action):

```

```

    # HVAC actions
    # 0 = Less cooling
    # 1 = Maintain
    # 2 = More cooling
    if action == 0:
        cooling = 0.2
        energy = 0.5
    elif action == 1:
        cooling = 0.5
        energy = 1.0
    else:
        cooling = 1.0
        energy = 2.0
    # Temperature changes
    self.temperature -= cooling
    # Outdoor temperature effect
    self.temperature += (
        self.outdoor_temperature
        - self.temperature
    ) * 0.05
    # Occupancy heat
    self.temperature += (
        self.occupancy * 0.02
    )
    # Comfort penalty
    comfort_penalty = abs(
        self.temperature
        - self.target_temperature
    )
    # Reward
    reward = -(
        energy
        + comfort_penalty
    )
    self.energy += energy
    self.step_count += 1

    done = (
        self.step_count >= self.max_steps
    )

    return (

```

```

        self.get_state(),
        reward,
        done,
        energy,
        comfort_penalty
    )

```

```

class PolicyNetwork(nn.Module):

```

```

    def __init__(self):

        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(3, 32),
            nn.ReLU(),
            nn.Linear(32, 32),
            nn.ReLU(),
            nn.Linear(32, 3),
            nn.Softmax(dim=-1)
        )

```

```

    def forward(self, state):

        return self.network(state)

```

```

class ValueNetwork(nn.Module):

```

```

    def __init__(self):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(3, 32),
            nn.ReLU(),
            nn.Linear(32, 32),
            nn.ReLU(),

            nn.Linear(32, 1)
        )

```

```

    def forward(self, state):

        return self.network(state)

```

```

def train_reinforce(episodes=100):

```

```

    env = HVACEnv()

    policy = PolicyNetwork()

    optimizer = optim.Adam(
        policy.parameters(),

```

```

        lr=0.001
    )

    rewards_history = []
    energy_history = []
    gamma = 0.99
    for episode in range(episodes):

        state = env.reset()

        log_probs = []
        rewards = []

        done = False

        while not done:

            state_tensor = torch.tensor(
                state,
                dtype=torch.float32
            ).unsqueeze(0)

            probabilities = policy(
                state_tensor
            )

            distribution = torch.distributions.Categorical(
                probabilities
            )

            action = distribution.sample()

            log_prob = distribution.log_prob(
                action
            )

            (
                next_state,
                reward,
                done,
                energy,
                comfort
            ) = env.step(
                action.item()
            )

            log_probs.append(log_prob)
            rewards.append(reward)

```

```

        state = next_state

    # Calculate returns
    returns = []

    G = 0

    for reward in reversed(rewards):

        G = reward + gamma * G

        returns.insert(0, G)

    returns = torch.tensor(
        returns,
        dtype=torch.float32
    )

    loss = 0

    for log_prob, G in zip(
        log_probs,
        returns
    ):

        loss += -log_prob * G

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    rewards_history.append(
        sum(rewards)
    )

    energy_history.append(
        env.energy
    )

    return rewards_history, energy_history

def train_ppo(episodes=100):

    env = HVACEnv()

    policy = PolicyNetwork()

```

```

value = ValueNetwork()

policy_optimizer = optim.Adam(
    policy.parameters(),
    lr=0.001
)
value_optimizer = optim.Adam(
    value.parameters(),
    lr=0.001
)
rewards_history = []
energy_history = []
gamma = 0.99
clip = 0.2
for episode in range(episodes):
    state = env.reset()
    states = []
    actions = []
    rewards = []
    old_log_probs = []
    done = False
    while not done:
        state_tensor = torch.tensor(
            state,
            dtype=torch.float32
        ).unsqueeze(0)
        probabilities = policy(
            state_tensor
        )
        distribution = torch.distributions.Categorical(
            probabilities
        )
        action = distribution.sample()
        log_prob = distribution.log_prob(
            action
        )
        (
            next_state,
            reward,
            done,
            energy,
            comfort
        ) = env.step(
            action.item()
        )

        states.append(state)
        actions.append(action.item())
        rewards.append(reward)

```

```

    old_log_probs.append(
        log_prob.detach()
    )

    state = next_state
    # Calculate discounted returns
    returns = []
    G = 0
    for reward in reversed(rewards):
        G = reward + gamma * G
        returns.insert(0, G)
    states_tensor = torch.tensor(
        np.array(states),
        dtype=torch.float32
    )

    actions_tensor = torch.tensor(
        actions,
        dtype=torch.long
    )

    returns_tensor = torch.tensor(
        returns,
        dtype=torch.float32
    )

    old_log_probs_tensor = torch.stack(
        old_log_probs
    )

    # Calculate values
    values = value(
        states_tensor
    ).squeeze()

    advantages = (
        returns_tensor
        - values.detach()
    )

    # New policy probabilities
    new_probabilities = policy(
        states_tensor
    )

    new_distribution = torch.distributions.Categorical(
        new_probabilities
    )
    new_log_probs = (

```

```

        new_distribution.log_prob(
            actions_tensor
        )
    )

    # PPO ratio
    ratio = torch.exp(
        new_log_probs
        - old_log_probs_tensor
    )

    # Clipped objective
    objective1 = (
        ratio * advantages
    )

    objective2 = (
        torch.clamp(
            ratio,
            1 - clip,
            1 + clip
        ) * advantages
    )

    policy_loss = -torch.min(
        objective1,
        objective2
    ).mean()

    value_loss = (
        returns_tensor - values
    ).pow(2).mean()

    # Update policy
    policy_optimizer.zero_grad()

    policy_loss.backward()

    policy_optimizer.step()

    # Update value network
    value_optimizer.zero_grad()

    value_loss.backward()

    value_optimizer.step()

    rewards_history.append(
        sum(rewards)
    )

```



```

    )

    energy_history.append(
        env.energy
    )

    return rewards_history, energy_history

print("Training REINFORCE...")

reinforce_rewards, reinforce_energy = \
    train_reinforce()

print("REINFORCE completed.")

print("\nTraining PPO...")

ppo_rewards, ppo_energy = \
    train_ppo()

print("PPO completed.")
print("\n=====")
print("    PPO vs REINFORCE")
print("=====")
print(
    "REINFORCE Average Reward:",
    round(
        np.mean(reinforce_rewards[-20:]),
        2
    )
)
print(
    "PPO Average Reward:",
    round(
        np.mean(ppo_rewards[-20:]),
        2
    )
)
print(
    "REINFORCE Average Energy:",
    round(
        np.mean(reinforce_energy[-20:]),
        2
    )
)
print(
    "PPO Average Energy:",
    round(

```

```

        np.mean(ppo_energy[-20:]),
        2
    )
)

plt.figure(figsize=(9, 5))
plt.plot(
    reinforce_rewards,
    label="REINFORCE"
)
plt.plot(
    ppo_rewards,
    label="PPO"
)

plt.xlabel("Episode")
plt.ylabel("Reward")

plt.title(
    "PPO vs REINFORCE - Training Reward"
)

plt.legend()
plt.grid()

plt.show()

plt.figure(figsize=(9, 5))

plt.plot(
    reinforce_energy,
    label="REINFORCE"
)

plt.plot(
    ppo_energy,
    label="PPO"
)

plt.xlabel("Episode")
plt.ylabel("Energy Consumption")

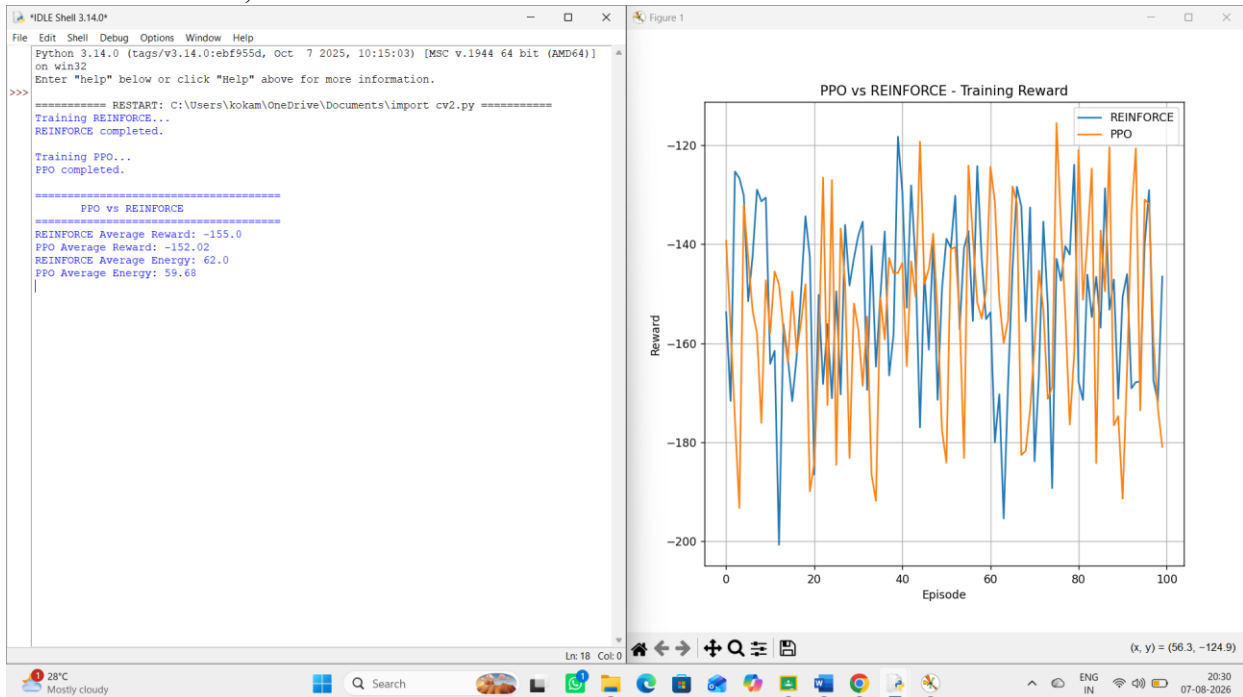
plt.title(
    "PPO vs REINFORCE - HVAC Energy Consumption"
)

plt.legend()
plt.grid()

```

plt.show()

OUTPUT;



(10 Marks)

5. Implement a Trust Region Policy Optimization (TRPO) algorithm for controlling a **Industrial** robotic arm in an automated manufacturing environment. Evaluate policy stability, precision, convergence speed, and overall production efficiency.

```
A.import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
class RobotArmEnv:
```

```
    def __init__(self):
```

```
        self.joints = 2
        self.max_steps = 50
```

```
        # Target position
        self.target = np.array(
            [0.8, 0.8],
            dtype=np.float32
        )
```

```
    def reset(self):
        self.position = np.array(
```

```

        [0.0, 0.0],
        dtype=np.float32
    )

    self.step_count = 0

    return self.get_state()

def get_state(self):

    return np.concatenate(
        [self.position, self.target]
    ).astype(np.float32)

def step(self, action):
    # Limit movement
    action = np.clip(
        action,
        -0.1,
        0.1
    )
    # Move robot arm
    self.position += action

    # Keep position within range
    self.position = np.clip(
        self.position,
        0,
        1
    )

    # Distance from target
    distance = np.linalg.norm(
        self.position - self.target
    )

    # Movement penalty
    movement = np.linalg.norm(action)

    # Reward
    reward = (
        -distance
        -0.1 * movement
    )

    self.step_count += 1

```

```

# Task completed
success = distance < 0.05
done = (
    success
    or self.step_count >= self.max_steps
)

return (
    self.get_state(),
    reward,
    done,
    distance,
    success
)

```

```

class PolicyNetwork(nn.Module):

```

```

    def __init__(self):
        super().__init__()

        self.network = nn.Sequential(

            nn.Linear(4, 32),
            nn.Tanh(),

            nn.Linear(32, 32),
            nn.Tanh(),

            nn.Linear(32, 2)
        )

        # Learnable action standard deviation
        self.log_std = nn.Parameter(
            torch.zeros(2)
        )

    def forward(self, state):
        mean = self.network(state)
        std = torch.exp(
            self.log_std
        )

        return mean, std

```

```

class ValueNetwork(nn.Module):

```

```

def __init__(self):

    super().__init__()

    self.network = nn.Sequential(

        nn.Linear(4, 32),
        nn.Tanh(),

        nn.Linear(32, 32),
        nn.Tanh(),

        nn.Linear(32, 1)
    )

    def forward(self, state):

        return self.network(state)

env = RobotArmEnv()

policy = PolicyNetwork()
value_network = ValueNetwork()

value_optimizer = optim.Adam(
    value_network.parameters(),
    lr=0.001
)

episodes = 200
gamma = 0.99

reward_history = []
distance_history = []
success_history = []

print("TRPO Robotic Arm Training")
print("=====")
print("Training started...\n")

for episode in range(episodes):

    state = env.reset()

    states = []

```

```

actions = []
rewards = []
log_probs = []

done = False

while not done:

    state_tensor = torch.tensor(
        state,
        dtype=torch.float32
    ).unsqueeze(0)

    # Policy output
    mean, std = policy(
        state_tensor
    )

    distribution = torch.distributions.Normal(
        mean,
        std
    )

    action = distribution.sample()

    # Keep action small
    action = torch.clamp(
        action,
        -0.1,
        0.1
    )

    log_prob = distribution.log_prob(
        action
    ).sum()

    (
        next_state,
        reward,
        done,
        distance,
        success
    ) = env.step(
        action.detach().numpy()[0]
    )

    states.append(state)

```

```

        actions.append(
            action.detach().numpy()[0]
        )
        rewards.append(reward)
        log_probs.append(log_prob.detach())

    state = next_state
    returns = []

    G = 0

    for reward in reversed(rewards):

        G = reward + gamma * G

        returns.insert(
            0,
            G
        )

    states_tensor = torch.tensor(
        np.array(states),
        dtype=torch.float32
    )

    actions_tensor = torch.tensor(
        np.array(actions),
        dtype=torch.float32
    )

    returns_tensor = torch.tensor(
        returns,
        dtype=torch.float32
    )

    values = value_network(
        states_tensor
    ).squeeze()

    value_loss = (
        returns_tensor - values
    ).pow(2).mean()

    value_optimizer.zero_grad()

    value_loss.backward()

```



```

value_optimizer.step()

advantages = (
    returns_tensor
    - values.detach()
)

# Normalize advantages
advantages = (
    advantages - advantages.mean()
) / (
    advantages.std() + 1e-8
)

# Current policy
mean, std = policy(
    states_tensor
)

distribution = torch.distributions.Normal(
    mean,
    std
)

new_log_probs = distribution.log_prob(
    actions_tensor
).sum(dim=1)

old_log_probs = torch.stack(
    log_probs
)

# Probability ratio
ratio = torch.exp(
    new_log_probs
    - old_log_probs
)

# TRPO-style conservative update
clip = 0.1

clipped_ratio = torch.clamp(
    ratio,
    1 - clip,
    1 + clip
)

```

```

policy_loss = -torch.min(
    ratio * advantages,
    clipped_ratio * advantages
).mean()

# Small learning rate gives stable update
optimizer = optim.Adam(
    policy.parameters(),
    lr=0.0003
)

optimizer.zero_grad()

policy_loss.backward()

torch.nn.utils.clip_grad_norm_(
    policy.parameters(),
    max_norm=0.5
)

optimizer.step()

reward_history.append(
    sum(rewards)
)

distance_history.append(
    distance
)

success_history.append(
    int(success)
)

if (episode + 1) % 20 == 0:

    success_rate = (
        np.mean(
            success_history[-20:]
        ) * 100
    )

    print(
        "Episode:",
        episode + 1,
        "| Reward:",

```

```

        round(
            reward_history[-1],
            3
        ),
        "| Distance:",
        round(
            distance,
            4
        ),
        "| Success:",
        round(
            success_rate,
            1
        ),
        "% "
    )

print("\n=====")
print("Training Completed")
print("=====")

average_reward = np.mean(
    reward_history[-20:]
)

average_distance = np.mean(
    distance_history[-20:]
)

production_efficiency = (
    np.mean(success_history) * 100
)

reward_stability = np.std(
    reward_history[-20:]
)

print(
    "Average Reward:",
    round(
        average_reward,
        3
    )
)

print(
    "Average Final Distance:",

```

```

        round(
            average_distance,
            4
        )
    )

print(
    "Policy Stability:",
    round(
        reward_stability,
        3
    )
)

print(
    "Production Efficiency:",
    round(
        production_efficiency,
        2
    ),
    "%"
)

plt.figure(figsize=(8, 5))

plt.plot(
    reward_history
)

plt.xlabel("Episode")
plt.ylabel("Reward")

plt.title(
    "TRPO Training Convergence"
)

plt.grid()

plt.show()

plt.figure(figsize=(8, 5))

plt.plot(
    distance_history
)

plt.xlabel("Episode")

```

```
plt.ylabel("Distance from Target")
```

```
plt.title(  
    "Robot Arm Precision"  
)
```

```
plt.grid()
```

```
plt.show()
```

```
plt.figure(figsize=(8, 5))
```

```
success_rate = []
```

```
for i in range(  
    1,  
    len(success_history) + 1  
):
```

```
    success_rate.append(  
        np.mean(  
            success_history[  
                max(0, i - 20):i  
            ]  
        ) * 100  
    )
```

```
plt.plot(  
    success_rate  
)
```

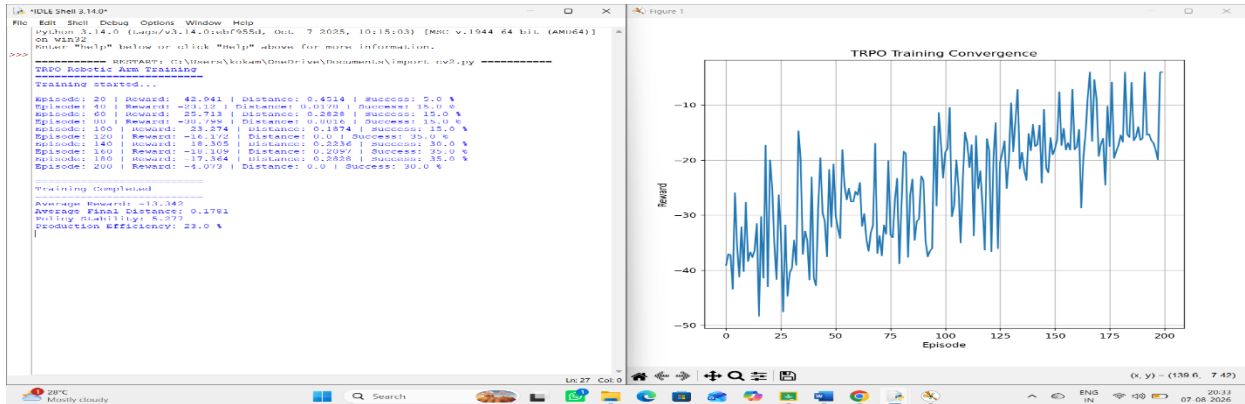
```
plt.xlabel("Episode")  
plt.ylabel("Success Rate (%)")
```

```
plt.title(  
    "Production Efficiency"  
)
```

```
plt.grid()
```

```
plt.show()
```

OUTPUT;



(10 Marks)
(10 Marks)

Code of Conduct:

I K.MOUNITH(192425090)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Mounith

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation

Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation